

1. 3-Schema ANSI/SPARC Architecture

Core Concept

Objective: Physical database se user applications ko decouple karna.

- 1. External Level (View Schema):** Multiple tailored views for end users. Unrelated tables hide hote hain (e.g., Student portal hides teacher salary).
- 2. Conceptual Level (Logical Schema):** Pure database structure ka global blueprint (entities, attributes, relationships, integrity constraints). Hardware independent.
- 3. Internal Level (Physical Schema):** Data physically hard disk par kaise stored hai (B+ Trees, hashing, clustering, file pointers, block allocation).

2. Data Independence

Exam Favourite

Ek schema level ko modify karne par uske higher level ko re-write na karna pade.

- Logical Data Independence:** Conceptual schema modify karne par External schema & user queries untouched rehti hain. (*Very Difficult to achieve*). Example: Adding column `DOB` to table does not break existing views.
- Physical Data Independence:** Internal schema (disk indexing, hashing, sorting) modify karne par Conceptual/External untouched rehti hain. (*Easily achieved*). Example: Changing Heap file to B+ Tree.

3. Traditional File System vs Modern DBMS

10-Marker Must

Evaluation Parameter	Traditional File System (OS Files)	Database Management System (DBMS)
Data Redundancy	Har department duplicate files maintain karta hai; immense disk waste.	Centralized schema; normalization through minimum redundancy.
Data Inconsistency	Ek file update hui, doosri purani reh gayi → Phantom data mismatch.	Single point updates & cascade rules; 100% mutual consistency.
Atomicity Guarantee	No rollback. Mid-transfer system crash leaves money deducted & lost.	WAL (Write-Ahead Logging) & 2-Phase Commit ensure All-or-Nothing.
Concurrent Access	File-level OS lock; reader-writer stalls or race conditions (lost update).	2PL & multi-version timestamping guarantee Serializability.
Security & Abstraction	Operating system coarse file permissions (rwx); no column masking.	Fine-grained column/row level SQL privileges (<code>GRANT SELECT</code>).

4. Database User Categories & DBA Roles

Viva & Theory

- Naive / Parametric:** Form GUI users (ATM, Railway booking, cashier).
- Application Programmers:** DML embedded code likhne wale (Java/C++ devs).
- Sophisticated Users:** Complex ad-hoc SQL / Analytical queries (Data Analysts).
- Specialized Users:** Non-relational apps (GIS spatial data, AI models).
- DBA (Database Administrator) Duties:** Schema definition, Storage structure & access path tuning, Granting authorization (DCL), Periodic backup & crash recovery.

5. DBMS Languages Taxonomy

Gate / AKTU

- DDL (Data Definition):** Schema create/alter. Auto-committed! Commands: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
- DML (Data Manipulation):** Tuples modify/retrieve.
 - Procedural (Low-level):* User specifies **WHAT** data and **HOW** to get it (Relational Algebra).
 - Declarative (High-level):* User specifies only **WHAT** data is needed (SQL, Relational Calculus). Commands: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- DCL (Control):** Permissions. `GRANT`, `REVOKE`.
- TCL (Transaction):** Integrity. `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

6. DBMS Tier Architectures

Architecture

- 1-Tier:** User UI, Client logic, aur Database ek hi physical machine par resident (Local SQLite, MS Access). Zero network overhead.
- 2-Tier:** Client PC par UI + App logic run hoti hai; Database remote server par hota hai via ODBC/JDBC. Fat client bottleneck.
- 3-Tier:** Client (Web Browser) → Application Server (Business Logic) → Database Server (Data Storage). Scalable, secure & enterprise standard.

7. Data Models Classification

Foundations

- 1. Object-Based Logical Models:** High level of abstraction. E-R (Entity-Relationship) Model and Object-Oriented Model.
- 2. Record-Based Logical Models:** Fixed format records.
 - Relational Model:* 2D Tables (Relations) connected by Foreign Keys (Codd, 1970).
 - Network Model:* Directed Graph (Arbitrary Many-to-Many via pointers).
 - Hierarchical Model:* Strict Tree structure (1:N parent-child, IMS).
- 3. Physical Data Models:** Frame memory, Unifying models.

1. Attributes Taxonomy & SymbolsER Elements Simple / Atomic: Cannot be broken further (Age, RollNo). Composite: Hierarchy of sub-parts (Name → First, Mid, Last). Single-Valued: Single item per tuple (Aadhar_No, DOB). Multivalued: Multiple items per entity (Phone_No, Skills). Symbol: Double Ellipse. Derived: Computed dynamically (Age = Current_Date - DOB). Symbol: Dashed Ellipse. Key Attribute: Primary identifier. Symbol: Underlined Text.	2. Cardinality & Participation ConstraintsMapping Logic Cardinality Ratios: 1:1 (One-to-One): Citizen ↔ Passport. 1:N (One-to-Many): Department → Employees. M:N (Many-to-Many): Students ↔ Courses. Participation Constraints: Total Participation (Double Line): Har entity relationship me mandatory appear hoti hai (e.g., Har Employee ka Dept hona mandatory). Partial Participation (Single Line): Kuch entities relation ke bina exist kar sakti hain (e.g., Har Professor Dean nahi hota).
3. Weak Entity Sets & Identifying RelationshipsCrucial Rule Definition: Aisa entity set jiske paas apna Primary Key form karne ke liye sufficient attributes na hon. Yeh identifying Owner Entity par depend karta hai. Weak Entity Symbol: Double Rectangle. Identifying Relationship: Double Diamond (Always 1:N from Owner to Weak, Total participation on Weak side). Partial Key / Discriminator: Dashed Underline attribute. Weak Entity PK = Owner PK + Partial Key (Discriminator) Real-World Canonical Example: - Owner: EMPLOYEE(Emp_ID, Name, Salary) - Weak: DEPENDENT(Dep_Name, Relation, Age) [Partial key: Dep_Name] - Resulting PK: {Emp_ID, Dep_Name} (Composite Primary Key).	
4. Relational Keys Master Hierarchy & Super Key MathFormula & GATE Super Key (SK): Any set of attributes jo relation ke har tuple ko uniquely identify kare. (May have redundant attributes). Candidate Key (CK): Minimal Super Key. Koi bhi proper subset super key nahi hona chahiye! Primary Key (PK): Database designer dwara choose kiya gaya single candidate key. CANNOT BE NULL! Alternate Key (AK): Candidate keys jo Primary Key choose hone se bache rahe (AK = CK - PK). Foreign Key (FK): Current table ka attribute jo kisi parent relation ke Primary Key ko refer karta hai. Compound / Composite Key: Key made of 2 or more attributes. Super Key Calculation Formula: If Relation has n attributes and single Candidate Key of size k: Total SKs = $2^{(n - k)}$. For multiple Candidate Keys: Apply Principle of Inclusion-Exclusion: $SK(A \cup B) = 2^{(n - A)} + 2^{(n - B)} - 2^{(n - A \cup B)}$.	
5. Extended ER (EER) Modeling FeaturesAdvanced ER Core Abstractions: Generalization: Bottom-up approach. Multiple lower entities ki common properties extract karke superclass banana (e.g., Car & Truck → Vehicle). Specialization: Top-down approach. Superclass ko specific distinguishing attributes ke basis par subclasses me split karna (e.g., Employee → Developer, QA). Aggregation: Relationship-as-Entity abstract karna taaki ternary relationship ko binary me clean model kiya ja sake. EER Constraints: Disjointness Constraint: <i>Disjoint ('d')</i>: Entity maximum 1 subclass ki member ho sakti hai (Mutually exclusive). <i>Overlapping ('o')</i>: Entity ek saath multiple subclasses me exist kar sakti hai. Completeness Constraint: <i>Total Specialization (Double Line)</i>: Har superclass entity kisi na kisi subclass me belong karegi. <i>Partial Specialization (Single Line)</i>: Entity bina kisi subclass me gaye superclass me reh sakti hai.	

1. The 7 ER-to-Relational Mapping Rules & Table Minimization Matrix

AKTU Top Scoring

ER Construct	Conversion / Mapping Rule	Min Tables	Resulting Relational Schema Structure
Strong Entity	Create dedicated table. Flatten composite attributes into atomic columns.	1 Table	STUDENT(<u>RollNo</u> , FName, LName, City, Marks)
Weak Entity	Table contains weak attributes + Owner PK as Foreign Key. Composite PK!	1 Table	DEPENDENT(<u>Emp_ID (FK)</u> , <u>Dep_Name</u> , Relation, Age)
1:1 (Both Total)	Merge both entity sets and relationship into a SINGLE unified table!	1 Table	CITIZEN_PASSPORT(<u>Aadhar_No</u> , Name, Pass_No, IssueDate)
1:1 (One Partial)	Maintain 2 tables. Place PK of Partial entity as Foreign Key on Total side.	2 Tables	EMP(<u>Emp_ID</u> , Name) & DEPT(<u>Dept_ID</u> , DName, Mgr_ID(FK))
1:N Relationship	NO new table! Put Primary Key of '1' side as Foreign Key into 'N' side table.	2 Tables	DEPT(<u>Dept_ID</u>) & EMP(<u>Emp_ID</u> , Name, Dept_ID(FK))
M:N Relationship	Create separate Join Table! PK = Composite union of both participating PKs.	3 Tables	STU(<u>Roll</u>), CRS(<u>CID</u>), ENROLL(<u>Roll(FK)</u> , <u>CID(FK)</u> , Grade)
Multivalued Attr	Create separate table. PK = (Entity PK + Multivalued Attribute).	+1 Table	STUDENT_PHONE(<u>RollNo(FK)</u> , <u>Phone_Number</u>)

2. Relational Data Model & The 4 Integrity Constraints

Rules

Domain Constraint:

Har attribute value uske defined valid atomic set me hona chahiye (e.g., $Age > 0$, $Marks \in [0, 100]$).

Key Constraint:

Relation ke koi bhi do distinct tuples identical Primary/Candidate Key share nahi kar sakte.

Entity Integrity:

Primary Key attribute value **KABHI BHI NULL NAHI** ho sakti! (Kyunki NULL identify nahi kar sakta).

Referential Integrity:

Foreign Key value ya to parent relation ke existing valid Primary Key se match karegi, ya fir **NULL** hogi. Violated on Delete/Update → Actions: CASCADE, SET NULL, RESTRICT.

3. Relational Algebra Operations Master Summary

Formal Operators

Operation	Symbol	Type	Behavior & Key Property	Example Syntax
Selection	$\sigma_p(R)$	Horizontal	Rows filter karta hai based on predicate condition p . Degree same, Cardinality $\leq R $.	$\sigma_{Salary > 50000}(EMPLOYEE)$
Projection	$\pi_A(R)$	Vertical	Columns filter karta hai. Automatically removes duplicate rows! Degree = $ A $.	$\pi_{Name, Salary}(EMPLOYEE)$
Cartesian Product	$R \times S$	Cross	Combines all tuples. Degree = $deg(R) + deg(S)$; Cardinality = $ R \times S $.	$STUDENT \times COURSE$
Natural Join	$R \bowtie S$	Join	Common attribute names par Cartesian product followed by equality selection and projection.	$EMP \bowtie DEPT$
Outer Joins	$\&LeftFullJoin;$, $\&LeftThreetimes;$, $\&RightThreetimes;$	Preserve	Unmatched dangling tuples ko retain karte hain padded with NULLs (Left, Right, Full).	$EMP \&LeftThreetimes; DEPT$
Division Operator	$R \div S$	Universal	Solves " FOR ALL / EVERY " queries. (Find students who took ALL math courses).	$\pi_{Roll, CID}(ENROLL) \div \pi_{CID}(MATH_CRS)$

4. Relational Calculus & Codd's Theorem

Theoretical Foundation

Tuple Relational Calculus (TRC):

Non-procedural. Focuses on filtering tuples: $\{ t \mid P(t) \}$. Where t is a tuple variable and $P(t)$ is a first-order formula using $\exists, \forall, \wedge, \vee, \neg$.

Domain Relational Calculus (DRC):

Focuses on filtering individual attribute domain variables: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$.

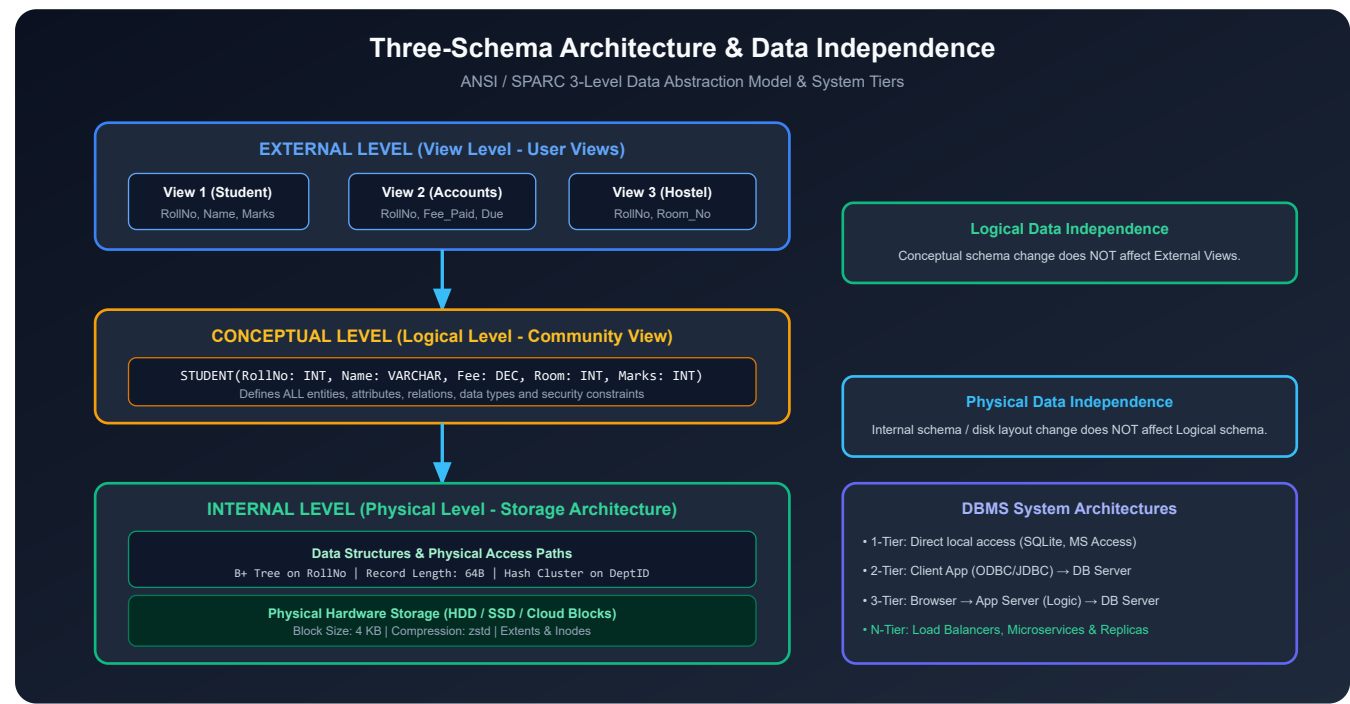
CODD'S THEOREM (Relational Completeness): Relational Algebra = Safe TRC = Safe DRC

A query language is **Relationally Complete** if it can express any query expressible in basic Relational Algebra. SQL is built upon this foundation!

Module 01: Database System Concepts & Architecture

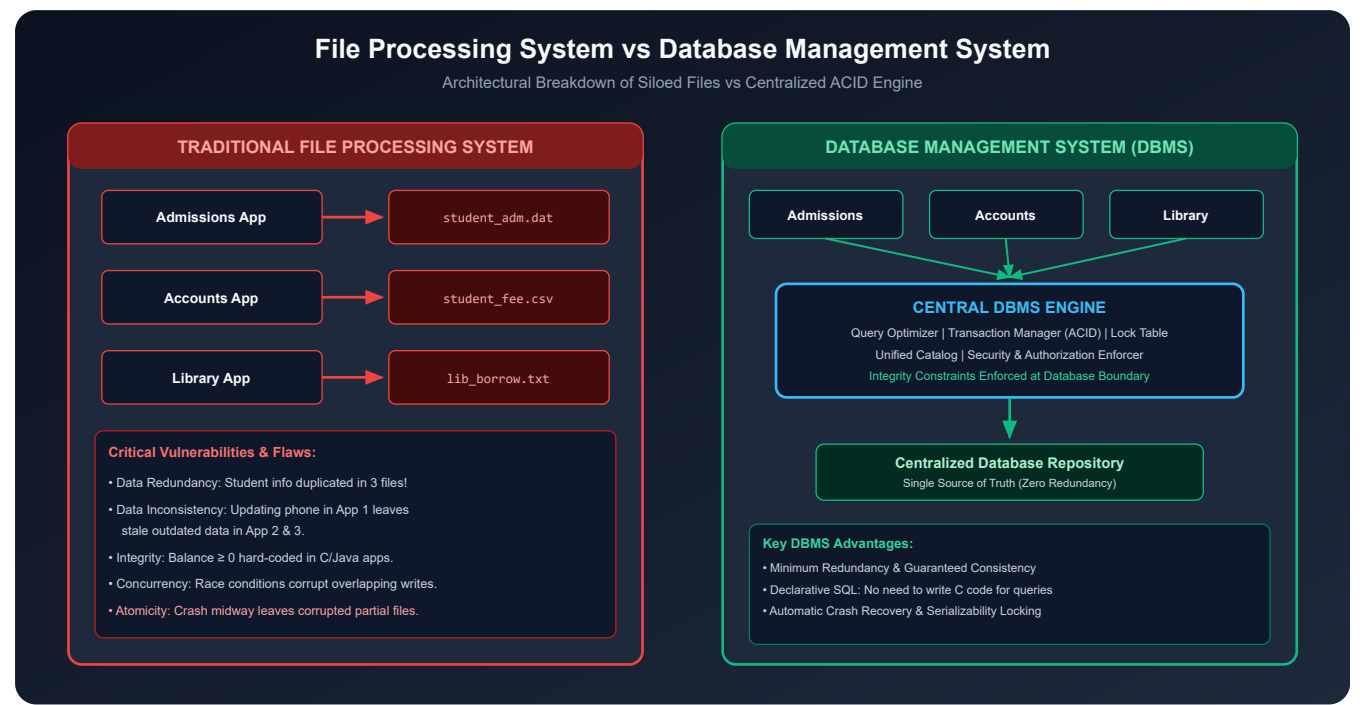
1. Three-Schema Architecture & Data Abstraction

ANSI/SPARC model decouples user applications from physical storage through three distinct abstraction boundaries: **External Level** (User Views), **Conceptual Level** (Logical Schema), and **Internal Level** (Physical Storage Layout).



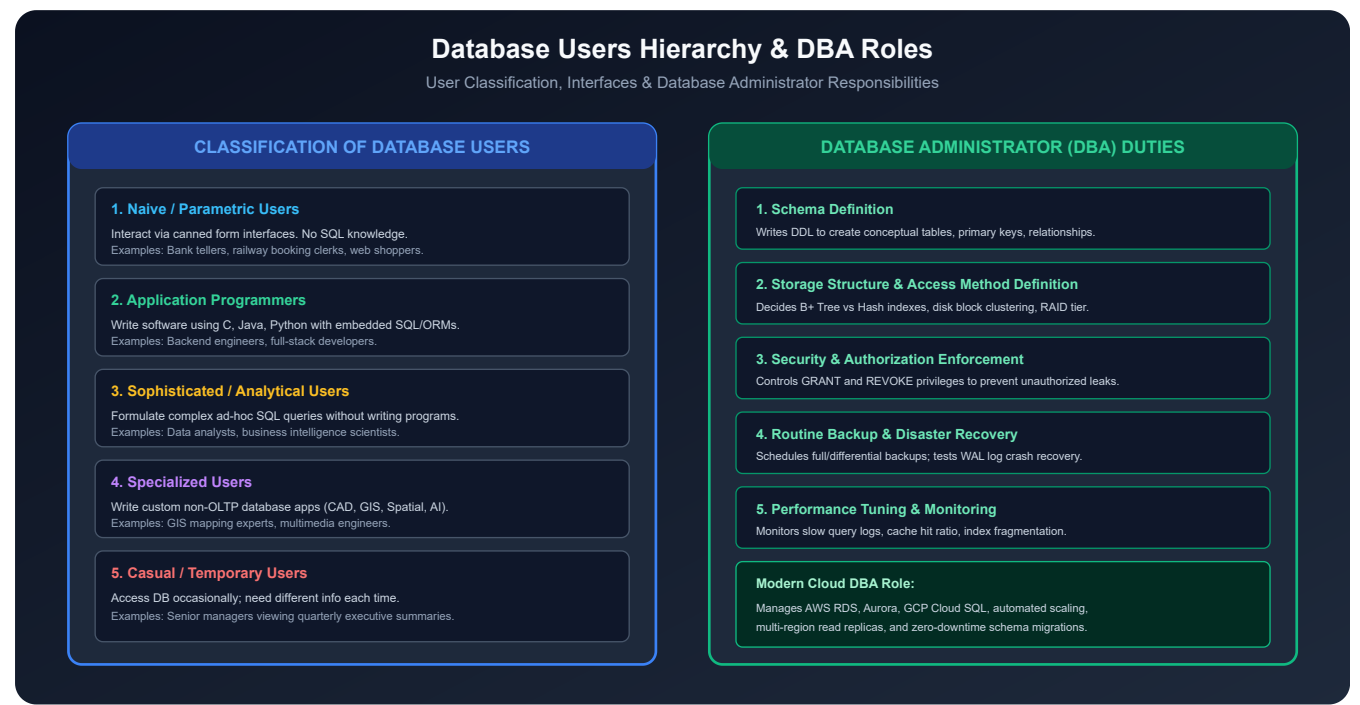
Module 02: File Processing System vs DBMS

1. Structural Comparison



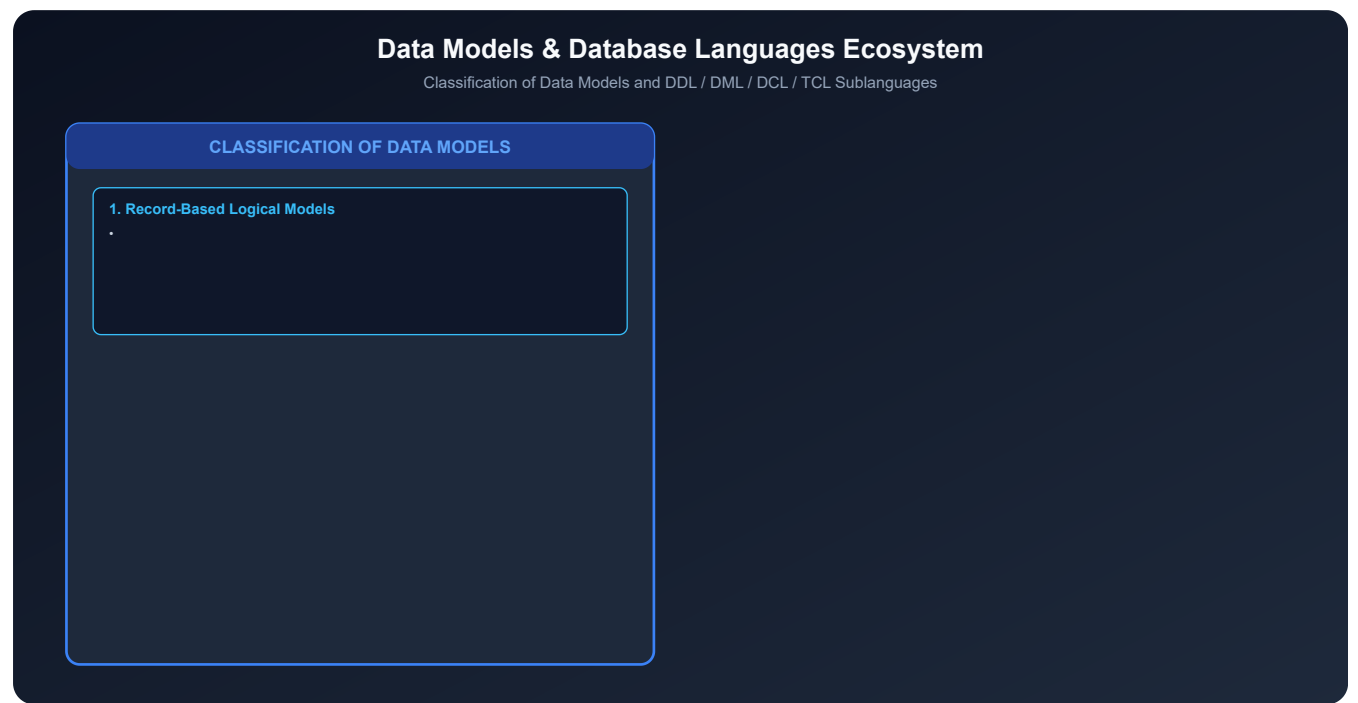
Module 03: Database Users & DBA Roles

1. User Classification & DBA Duties Overview



Module 04: Data Models and Languages

1. Data Models Classification



Relational Model: Data organized in 2D tables (relations). • **Network Model (CODASYL):** Records connected via pointers/graphs. • **Hierarchical Model:** Tree structure, parent-child segments (IMS). Standard across industry: Relational dominates. 2. Object-Based Data Models • **Object-Oriented (OODBMS):** Objects, classes, methods, inheritance. • **Object-Relational (ORDBMS):** Relational + complex user-defined types (PostgreSQL). 3. Semi-Structured Models (NoSQL) • JSON documents, BSON, XML trees (MongoDB, DynamoDB). Schema-on-read flexibility without predefined DDL tables. 4. Physical Data Models Low-level record formats, byte-offsets, block headers. DATABASE LANGUAGES (SQL SUBSYSTEMS) DDL (Data Definition Language) Defines schema structure & writes metadata to Data Dictionary. Commands: CREATE, ALTER, DROP, TRUNCATE, RENAME DML (Data Manipulation Language) Accesses and manipulates user data inside tables. • Procedural DML: User specifies WHAT and HOW (Relational Alg) • Non-Procedural (Declarative) DML: User specifies WHAT only (SQL) Commands: SELECT, INSERT, UPDATE, DELETE DCL (Data Control Language) Manages security privileges and access rights. Commands: GRANT, REVOKE TCL (Transaction Control Language) Controls transaction boundaries and atomicity rollback. Commands: COMMIT, ROLLBACK, SAVEPOINT

2. Procedural vs Non-Procedural DML (AKTU Essential)

Feature	Procedural DML	Non-Procedural (Declarative) DML
Query Specification	Specifies WHAT data is needed AND HOW to retrieve it	Specifies WHAT data is needed WITHOUT HOW
Execution Control	User/Programmer controls traversal algorithm	DBMS Query Optimizer decides optimal physical plan
Complexity	High programming overhead	Simple, intuitive, high-level

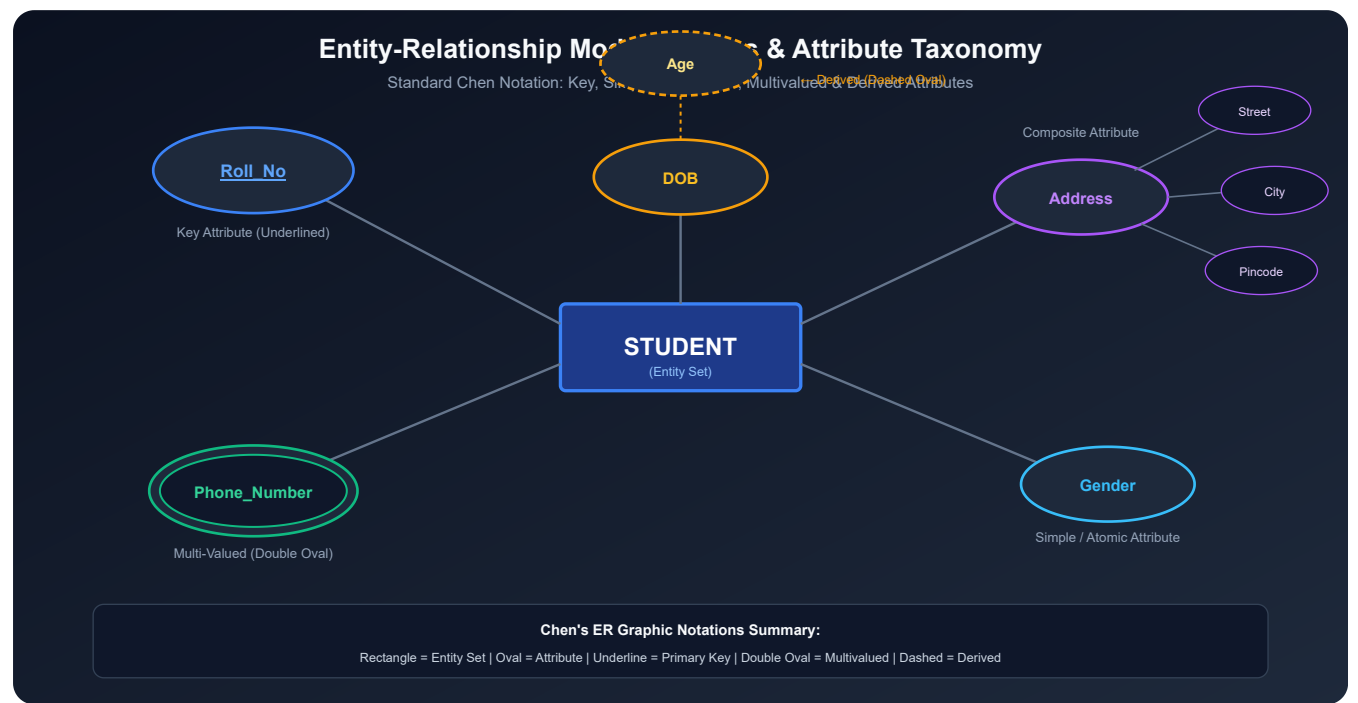
Primary Examples	Relational Algebra, Network/Hierarchical DML	SQL, Relational Calculus
------------------	--	--------------------------

3. Database Sublanguages (DDL / DML / DCL / TCL)

- **DDL:** CREATE, ALTER, DROP, TRUNCATE (Updates System Catalog Metadata)
- **DML:** SELECT, INSERT, UPDATE, DELETE (Manipulates Table Tuples)
- **DCL:** GRANT, REVOKE (Access Control Privileges)
- **TCL:** COMMIT, ROLLBACK, SAVEPOINT (Transaction State Control)

Module 05: ER Model Entities & Attributes

1. Visual Attribute Taxonomy



2. Core Attribute Categories

Attribute Type	Chen Notation	Key Characteristic	Example
Simple / Atomic	Single Oval	Indivisible atomic value	Gender
Composite	Hierarchical Ovals	Divisible into sub-attributes	Address → Street, City, Pin
Key Attribute	Underlined Text	Uniquely identifies entity	<u>Roll_No</u>
Multi-Valued	Double Oval	Entity can hold multiple values	Phone_Number
Derived	Dashed Oval	Calculated on-the-fly from stored data	Age = Today - DOB

Module 06: Relationships, Cardinalities & Weak Entities

1. Architectural Architecture Overview



2. Mapping Cardinalities & Participation Constraints

Ratio	Constraint Meaning	Total vs Partial Participation	Example
1 : 1	Citizen has at most 1 Passport	Passport has Total Participation	Citizen ↔ Passport
1 : N	1 Dept employs N Employees	Employee has Total (Must have a Dept)	Department → Employee
M : N	M Students enroll in N Courses	Both sides can be Partial/Total	Student ↔ Course

3. Weak Entity Set Rules (Exam Must-Know)

1. **Double Rectangle:** Weak entity container.

2. **Double Diamond:** Identifying relationship with Strong owner entity.

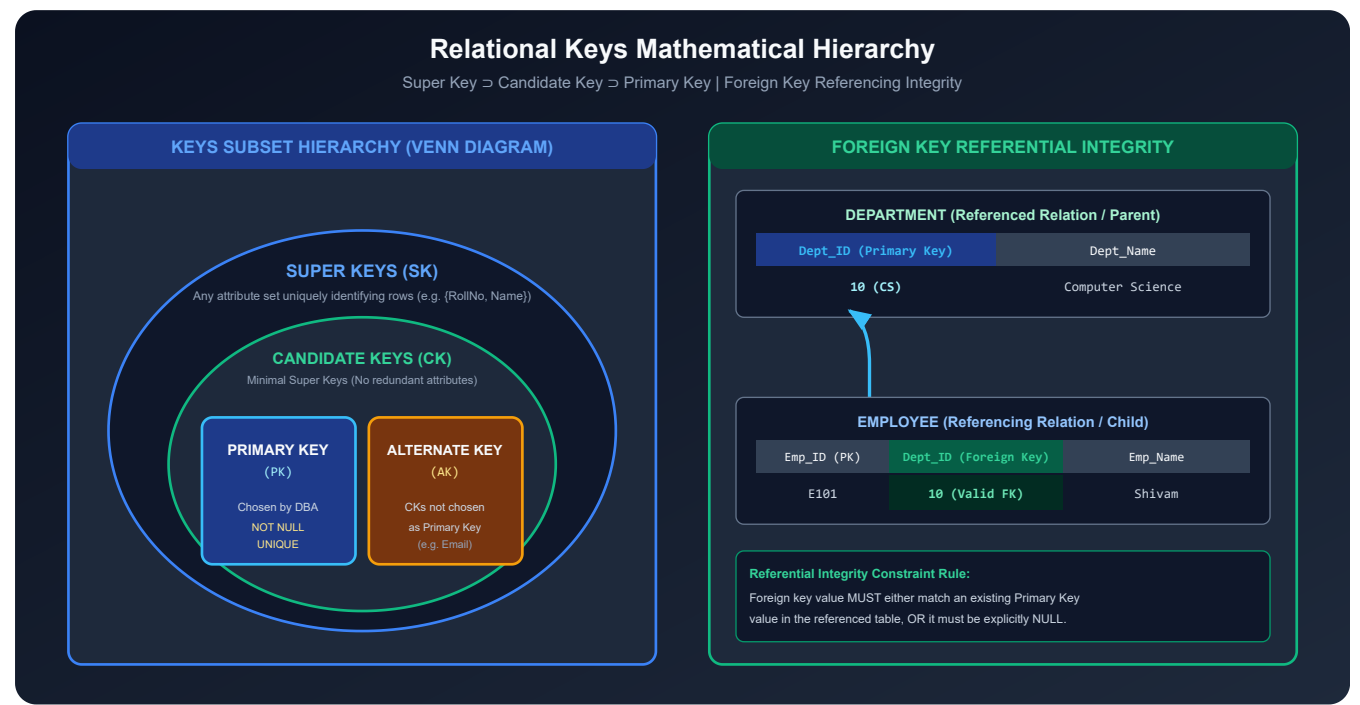
3. **Dashed Underline:** Partial key / Discriminator.

4. **Double Line:** Always total participation.

5. **Composite PK:** PK(Weak) = PK(Owner) + Discriminator.

Module 07: Relational Keys in Depth

1. Keys Hierarchy & Foreign Key Architecture



2. Comprehensive Key Types

Key Type	Minimality	NULL Permitted?	Purpose
Super Key	No (Can contain extra attributes)	Yes	Uniquely identifies tuples
Candidate Key	Strictly Minimal	No (usually)	Eligible candidate to become PK
Primary Key	Strictly Minimal	Strictly NEVER NULL	Central unique row identifier
Alternate Key	Strictly Minimal	Yes	Candidate keys not picked as PK
Foreign Key	Depends on target PK	YES (Unless NOT NULL constraint)	Cross-table referential integrity

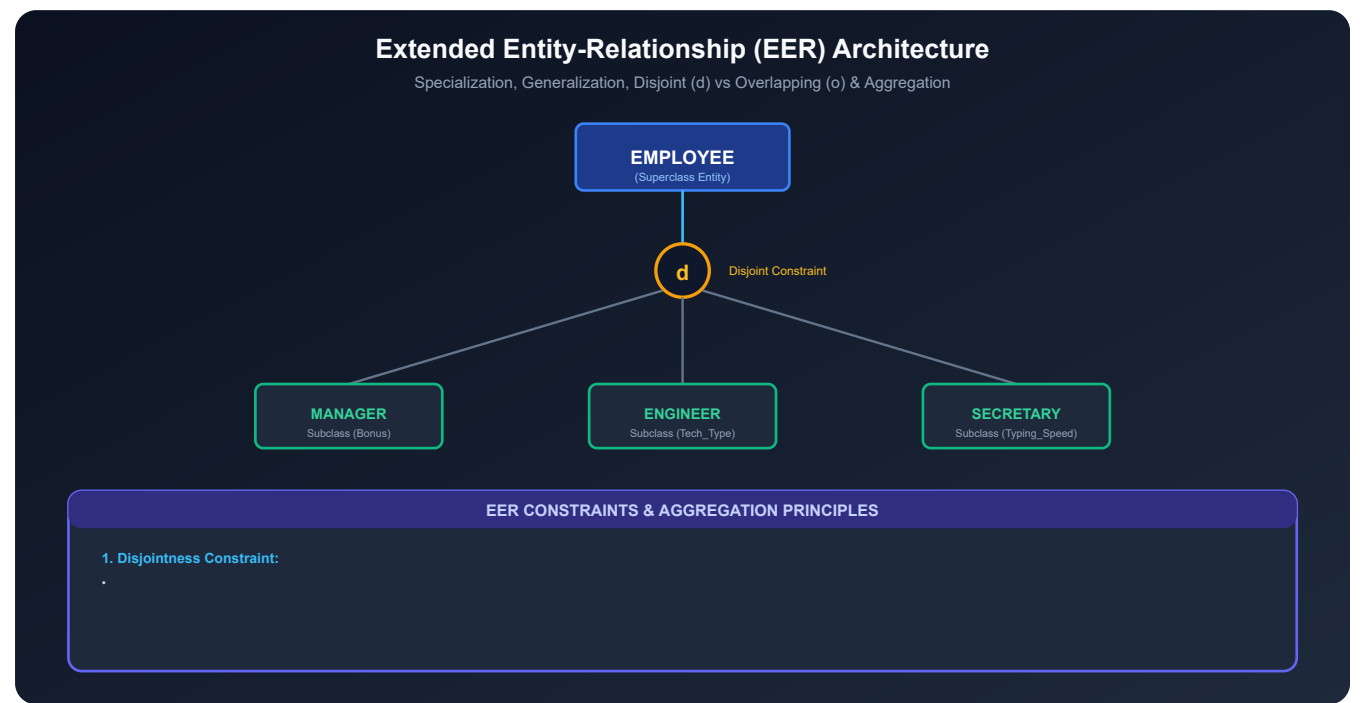
3. Solved Super Key Formula

If a relation has n attributes and a candidate key of size k attributes, the total number of super keys containing this candidate key is:

Total Super Keys = $2^{(n - k)}$

Module 08: Extended ER Features (EER)

1. EER Architecture & Taxonomy



Disjoint (d): Entity can belong to at most ONE subclass (e.g. Can't be Manager AND Secretary simultaneously). • **Overlapping (o):** Entity can belong to MULTIPLE subclasses (e.g. Employee can be Teacher AND Student). 2. Completeness Constraint: • **Total (Double line):** Every superclass entity MUST belong to at least one subclass. • **Partial (Single line):** Some superclass entities may not belong to any subclass. **Aggregation:** Treating a relationship between entity sets as a higher-level composite entity so it can enter a secondary relationship.

2. Specialization vs Generalization

Feature	Specialization	Generalization
Approach	Top-Down Process	Bottom-Up Process
Objective	Identify distinguishing subclass properties	Generalize common superclass attributes
Inheritance	Subclass inherits all superclass attributes	Common attributes extracted into superclass

3. Core EER Constraints

- **Disjoint (d):** Entity can belong to at most 1 subclass.
- **Overlapping (o):** Entity can simultaneously belong to multiple subclasses.
- **Total (Double Line):** Entity must belong to at least 1 subclass.
- **Partial (Single Line):** Entity may not belong to any defined subclass.
- **Aggregation:** Abstracts a relationship into a higher-level composite entity.

Module 09: ER-to-Relational Mapping & Table Reduction

1. The 7 Conversion Rules

ER-to-Relational Schema Conversion Rules

The 7 Fundamental Mapping Algorithms and Relational Table Minimization

Rule 1: Strong Entity Set → 1 Table
All simple attributes become columns. Composite attributes flattened.

STUDENT(RollNo, Name, Street, City, State, DOB)

Rule 2: Weak Entity Set → 1 Table (With Owner PK)
Weak attributes + Owner PK as Foreign Key. PK is Composite.

DEPENDENT(Emp_ID (FK), Dep_Name, Relation, Age)

Rule 3: Binary 1:1 Relationship
• Foreign Key approach: Add PK of either side as FK (prefer Total side).
• Merged Table approach: If BOTH sides Total, combine into 1 table!

CITIZEN(Aadhar_No, Name) | PASSPORT(Pass_No, Issue, Aadhar_No(FK))

Rule 4: Binary 1:N Relationship → Foreign Key on 'N' Side
Do NOT create separate table! Place PK of '1' side into 'N' side table.

DEPT(Dept_ID, DName) [1 Side]
EMPLOYEE(Emp_ID, EName, Dept_ID (FK)) [N Side]

Rule 5: Binary M:N Relationship → Dedicated Join Table
Requires separate relationship table! Primary Key is composite.

STUDENT(RollNo, Name) | COURSE(Course_ID, Title)
ENROLLED(RollNo(FK), Course_ID(FK), Grade, Enroll_Date)

Rule 6: Multivalued Attribute → Separate Table
Stores Entity PK + Multivalued Attribute. PK is composite pair.

STUDENT(RollNo, Name)
STUDENT_PHONE(RollNo(FK), Phone_Number)

Table Minimization Principles (Exam Formula):

- 1:1 with Both Total → 1 Table (Merge both entities + relation)
- 1:1 with One Partial → 2 Tables (Foreign key on Total side)
- 1:N Relationship → 2 Tables (Merge relation into 'N' side)
- M:N Relationship → 3 Tables (Entity A, Entity B, and Join Table)
- Weak Entity → 2 Tables (Merged identifying relationship)

Golden Rule: Never create separate tables for 1:N relationships; always merge with the 'N' side entity table!

2. Quick Minimization Formula (AKTU Numerical)

Relationship Type	Participation	Minimum Relational Tables
1 : 1	Both Total Participation	1 Table (Fully Merged)
1 : 1	One Total, One Partial	2 Tables (FK on Total side)
1 : N	Any Participation	2 Tables (Merge relation into 'N' side)
M : N	Any Participation	3 Tables (Table A, Table B + Join Table)
Weak Entity	Total Participation (Standard)	2 Tables (Owner Table + Weak Table)
Multivalued Attribute	1 Multivalued column	2 Tables (Entity Table + Attribute Table)

Module 10: Relational Data Model & Integrity Constraints

1. Relation Anatomy & Constraints Architecture

RELATIONAL DATA MODEL & INTEGRITY CONSTRAINTS

Relation Anatomy: Tuples, Attributes, Domains, Degree, Cardinality & The 4 Constraints

RELATION ANATOMY: STUDENT(RollNo, Name, Marks)

RollNo (PK)	Name	Marks
101	Akhil Sharma	92
102	Rahul Gupta	85
103	Pooja Verma	95

- Degree (Arity): Number of attributes / columns = 3
- Cardinality: Number of tuples / rows = 3
- Domain: Permitted set of atomic values (e.g. Marks $\in [0, 100]$)

Update Operations & Constraint Checks:

- INSERT: Can violate Domain, Key, Entity, or Referential integrity.
- DELETE: Can only violate Referential Integrity (if child references it).
- UPDATE: Combination of Delete old + Insert new.

DBMS automatically rejects transactions violating constraints!

THE 4 RELATIONAL INTEGRITY CONSTRAINTS

1. Domain Constraint

Every attribute value must be atomic and within defined data type.
Example: Age cannot be "ABC" or a set of values.

2. Key Constraint

No two distinct tuples can have identical candidate key values.
 $t_1[\text{Candidate_Key}] \neq t_2[\text{Candidate_Key}]$

3. Entity Integrity Constraint

Primary Key attributes can NEVER be NULL!
If PK is NULL, individual tuples cannot be identified or indexed.
 $t[\text{Primary_Key}] \neq \text{NULL}$

4. Referential Integrity Constraint

Foreign key value in referencing relation must either:
(a) Match an existing PK in referenced relation, OR (b) Be NULL.
Violation Triggers: ON DELETE CASCADE / SET NULL / RESTRICT

2. The 4 Fundamental Integrity Constraints

Constraint	Formal Rule	Critical Rule
Domain Constraint	Attribute value $\in \text{dom}(A)$	Must be atomic (indivisible)
Key Constraint	$t_1[K] \neq t_2[K]$ for distinct tuples	Uniqueness across all tuples
Entity Integrity	$t[\text{Primary_Key}] \neq \text{NULL}$	Primary key is NEVER NULL
Referential Integrity	Foreign Key $\rightarrow$ Parent Primary Key	Must match valid parent PK OR be NULL

3. Referential Action Triggers

- CASCADE: Auto-deletes child rows upon parent deletion.
- SET NULL: Sets child FK column to NULL.
- RESTRICT: Rejects parent modification if referenced by active child tuples.

Module 11: Relational Algebra & Calculus

1. Relational Operations Taxonomy

RELATIONAL ALGEBRA OPERATIONS & QUERY LANGUAGES

Fundamental Operations (σ , π , $\times$, $\cup$, $-$), Joins ($\bowtie$), Division ($\div$) and Calculus

SELECTION: $\sigma_{\{Marks > 80\}}(R)$

Roll	Name	Marks	
101	Akhil	92	✓ SELECTED
102	Rahul	75	(Filtered out)
103	Pooja	88	✓ SELECTED

Horizontal Filtering of Tuples
Degree remains same; Cardinality $\leq$ original

PROJECTION: $\pi_{\{Roll, Name\}}(R)$

Roll	Name	Marks
101	Akhil	
102	Rahul	Dropped
103	Pooja	

Vertical Filtering of Columns
Auto-eliminates duplicate tuples!

JOIN OPERATIONS ($R \bowtie S$)

- Natural Join ($R \bowtie S$):**
Equi-join on common attributes; duplicate column dropped.
- Left Outer Join ($R \bowtie\!\!\!\rhd S$):**
All left tuples kept; unmatched right padded with NULL.
- Right Outer Join ($R \bowtie\!\!\!\lhd S$):**
All right tuples kept; unmatched left padded with NULL.
- Full Outer Join ($R \bowtie\!\!\!\lhd\!\!\!\rhd S$):**
All tuples from both sides kept with NULL padding.

DIVISION OPERATOR ($\div$) & RELATIONAL CALCULUS (TRC / DRC)

Division Operator ($R \div S$) - "FOR ALL" Queries

Used for queries with condition "Find entities associated with ALL..."
Example: "Find students enrolled in ALL courses taught by Prof. Rao"
Formula: $R \div S = \pi_{\{R-S\}}(R) - \pi_{\{R-S\}}((\pi_{\{R-S\}}(R) \times S) - R)$
If $R(A, B)$ has degree 2 and $S(B)$ has degree 1 $\rightarrow (R \div S)$ has degree 1 (A).
Highly popular in GATE and AKTU 10-mark examinations!

Relational Calculus: Declarative Formal Languages

- TRC (Tuple Relational Calculus): $\{ t \mid P(t) \}$
Quantifiers: $\exists$ (There exists), $\forall$ (For all). Safety rule restricts infinite sets.
- DRC (Domain Relational Calculus): $\{ \langle x1, x2 \rangle \mid P(x1, x2) \}$
Variables range over attribute domain values instead of whole tuples.
- Codd's Theorem: Relational Algebra $\equiv$ Safe TRC $\equiv$ Safe DRC!

2. Fundamental vs Derived Operations

Operation	Notation	Type	Core Definition
Selection	$\sigma_p(R)$	Fundamental	Horizontal filtering of tuples
Projection	$\pi_{cols}(R)$	Fundamental	Vertical filtering of columns (auto-deduplication)
Cartesian Product	$R \times S$	Fundamental	Pairwise tuple combinations
Set Union	$R \cup S$	Fundamental	Combines union-compatible tables
Set Difference	$R - S$	Fundamental	Tuples in R but not in S
Natural Join	$R \bowtie S$	Derived	Equi-join on shared attributes
Division	$R \div S$	Derived	Answers "ALL" universally quantified queries

3. Codd's Theorem

Relational Completeness: Relational Algebra $\equiv$ Safe Tuple Relational Calculus (TRC) $\equiv$ Safe Domain Relational Calculus (DRC). SQL is built on top of these foundations.

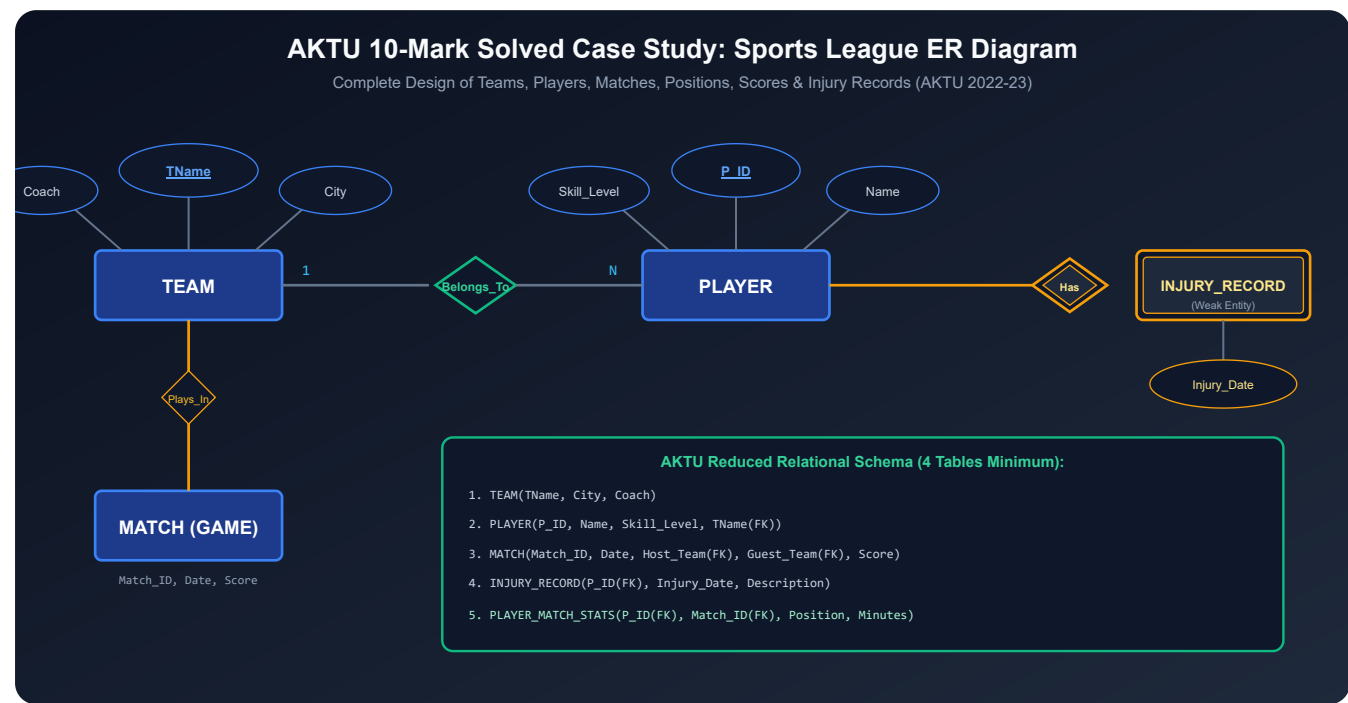
Module 12: Unit 1 AKTU PYQs & Solved Case Studies

Unit 1: Exam Solutions

AKTU BCS501

Solved Case Studies

1. Solved 10-Mark Case Study: Sports League ER Design



2. Solved Relational Algebra Queries (AKTU Exam)

Schema: EMPLOYEE(Emp_ID, Name, Salary, Dept_ID), DEPARTMENT(Dept_ID, DName, Location)

1. Find employees in 'Research':
 $\pi_{Name}(\sigma_{DName = 'Research'}(EMPLOYEE \bowtie DEPARTMENT))$

2. Find employees with Salary > 50,000 in 'Noida':
 $\pi_{Name}(\sigma_{Salary > 50000 \wedge Location = 'Noida'}(EMPLOYEE \bowtie DEPARTMENT))$

3. Find departments having ZERO employees:
 $\pi_{DName}(DEPARTMENT) - \pi_{DName}(EMPLOYEE \bowtie DEPARTMENT)$

3. Core Interview Q&A

Q: Why does Primary Key strictly prohibit NULL?

A: Entity Integrity Constraint demands that every tuple must have an unambiguous, non-null physical identity. In SQL three-valued logic, NULL = NULL evaluates to UNKNOWN, making two NULL keys mathematically indistinguishable.